Netty 面试题速记通关版 _ 面试刷题 mianshiya.com

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*

什么是 Netty，它在网络编程中解决了什么问题？

Netty 是一个基于 NIO 的高性能网络应用框架，用来快速开发可维护的高性能、高可扩展性的协议服务器与客户端。很多人觉得直接用 Java 原生 NIO 也能写网络通信，但那套 API 太难用了，得自己处理各种底层细节，比如缓冲区管理、半包粘包、连接状态跟踪，稍不注意就出 bug。

它真正解决的是“如何让开发者专注业务逻辑”这件事。传统 NIO 编程容易搞出内存泄漏、线程死锁，还得手动实现编解码器。Netty 把这些脏活累活都封装好了，提供统一的事件驱动模型，把网络读写、连接建立、异常处理等变成一个个 handler，像搭积木一样拼装。

比如你要做一个即时通讯服务，用 Netty 可以轻松集成 WebSocket 协议，配合 Redis 实现集群消息广播；或者在微服务里做自定义 RPC 协议传输，像 Dubbo 底层就用了 Netty 作为通信层。

1）基于主从 Reactor 多线程模型，一个 Boss 线程组接连接，多个 Worker 线程处理 IO 读写
2）自带 ChannelPipeline 机制，支持自定义编解码、心跳检测、流量整形
3）零拷贝体现在堆外内存 DirectBuffer 和 CompositeByteBuf 合并缓冲区时不复制数据

代码上基本就是启动类 + ChannelHandler 组合：

```
new ServerBootstrap()
    .group(bossGroup, workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new ChannelInitializer<SocketChannel>() {
        protected void initChannel(SocketChannel ch) {
            ch.pipeline().addLast(new StringDecoder());
            ch.pipeline().addLast(new EchoHandler()); // 业务处理器
        }
    });
```

说说你知道的几种 I/O 模型

Unix 系统里处理网络 I/O，绕不开五种经典模型，它们本质是描述数据在内核态和用户态之间搬移的过程差异。

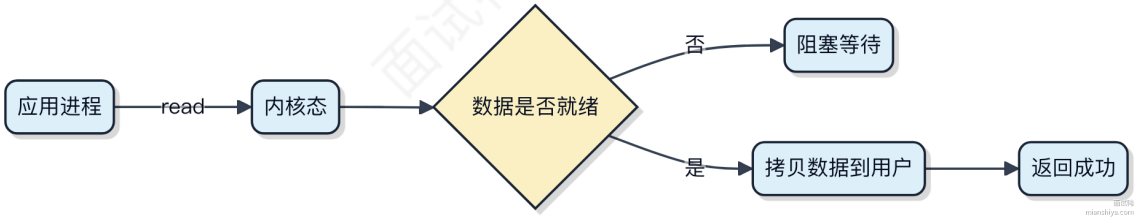
1）阻塞 I/O 最常见，比如 Java 传统 Socket 编程。调用 read 时，线程卡在那里，直到数据从网卡到达并拷贝到用户缓冲区。期间 CPU 干不了别的，**同步阻塞**，简单但并发上不去。

2）非阻塞 I/O 会把 socket 设成 NONBLOCK。每次 read 直接返回，有数据就读，没数据就报 EAGAIN。需要轮询，CPU 空转严重，实际很少直接用。

3）I/O 多路复用才是重点。select、poll、epoll 都属于这一类。一个线程能监听多个 socket 的事件。比如 epoll_ctl 注册关注的 fd，epoll_wait 阻塞等待，哪个连接来了数据或可写了，就通知你去处理。Nginx、Redis、Netty 都靠它扛住几万并发。

4）信号驱动 I/O 用 SIGIO 信号触发，内核收到数据后发信号，应用再读。异步通知，但信号处理复杂，用的人少。

5）真正的异步 I/O，POSIX 的 aio_read，Linux 的 io_uring。发起读请求后立刻返回，数据由内核完整准备好并拷贝到用户空间后，再回调你的函数。整个过程不阻塞，**完全异步**，但编程模型复杂，glibc 的 aio 实现还不成熟，Java 的 AIO 实际落地不多。



主流高性能服务基本都选 epoll 这条路，代码稍微复杂点，但资源利用率最高。

说一下 Netty 的应用场景？

Netty 本质是个高性能的网络编程框架，把 NIO 那套复杂的 API 给封装得特别顺手。它真正发力的地方是需要处理大量并发连接、低延迟通信的场景。

1）微服务内部通信首选。像 Dubbo、gRPC 这些 RPC 框架底层都用 Netty 做传输层，因为能轻松扛住几万甚至十几万的并发连接，吞吐量也稳。

2）即时通讯和消息推送绕不开它。比如做聊天室、直播弹幕、APP 推送服务，客户端长连接一多，传统 BIO 直接崩，Netty 的事件驱动模型压根不经过线程阻塞，一条连接一个 Channel，资源占用少得多。

3）网关类中间件也常见。Spring Cloud Gateway 虽然默认用 Reactor Netty，但底层还是 Netty 的能力，用来做请求路由、协议转换、限流熔断，性能损耗小。

4）游戏服务器后端经常拿它打底。帧同步、状态广播这些高频操作，要求毫秒级响应，Netty 配合自定义协议编解码，能把延迟压到很低。

写个最简示例，启动一个 TCP 服务端也就十来行代码：

```
EventLoopGroup boss = new NioEventLoopGroup(1);
EventLoopGroup worker = new NioEventLoopGroup();
ServerBootstrap b = new ServerBootstrap();
b.group(boss, worker)
 .channel(NioServerSocketChannel.class)
 .childHandler(new ChannelInitializer<SocketChannel>() {
     public void initChannel(SocketChannel ch) {
         ch.pipeline().addLast(new SimpleChannelInboundHandler<ByteBuf>() {
```

```
                protected void channelRead0(ChannelHandlerContext ctx, ByteBuf msg) {
                    // 处理读取数据
                }
            });
        }
    });
b.bind(8080).sync();
```

这种简洁性和可控性，让 Netty 在高并发网络通信领域几乎成了事实标准。

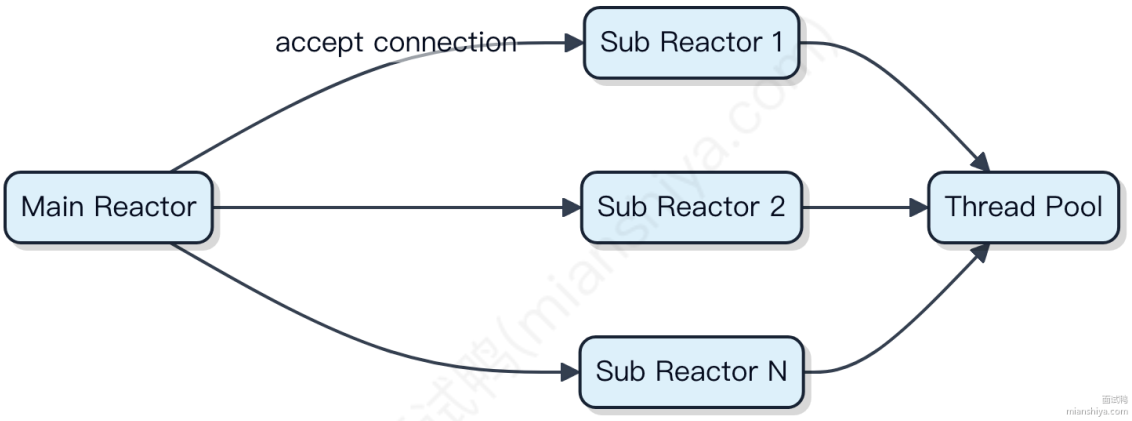
介绍一下 Reactor 线程模型？

Reactor 模型本质是事件驱动的设计，用少量线程处理大量并发连接，避免为每个连接创建线程带来的资源开销。Java NIO 的 Selector 就是典型实现。

1）单 Reactor 单线程
一个线程负责监听事件、分发请求、处理 I/O 和业务逻辑。适合低并发场景，比如 Redis 主线程就是这种模式。代码结构简单，但 CPU 密集任务会阻塞整个流程。

2）单 Reactor 多线程
Reactor 线程只做事件监听和分发，具体业务交给线程池处理。这样 I/O 和计算解耦，Netty 默认就是这种模型。注意 Reactor 仍只有一个，注册、事件分发都在同一线程。

3）主从 Reactor 多线程
多个 Reactor 分工协作，通常一个 Main Reactor 负责 Accept 新连接，把建立好的连接注册到多个 Sub Reactor 上，由它们处理读写事件。Nginx 和 Netty 都支持这种模式，能更好利用多核。



实际选型看场景：轻量通信用单线程，高并发网络服务上主从多线程。Netty 通过 `EventLoopGroup` 配置就能切换模式，灵活性很强。

简单说说 Netty 的零拷贝机制？

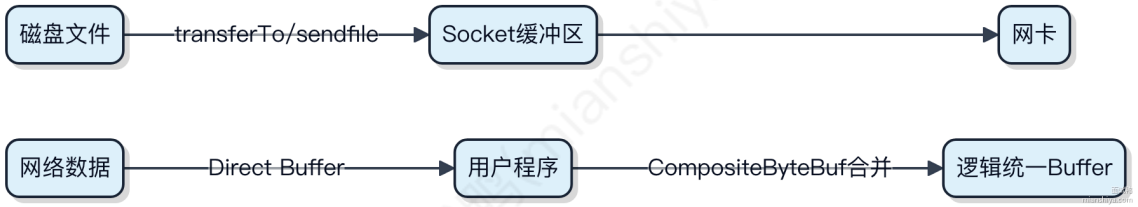
Netty 的零拷贝不是操作系统层面的 `zero-copy` ，而是应用层通过 **CompositeByteBuf** 和 **FileRegion** 等设计，减少数据在 JVM 堆内不必要的复制和内存占用。

1）比如接收多个 Buffer 拼接成完整消息时，传统做法是分配新内存，把各个片段拷贝过去。Netty 用 CompositeByteBuf 把多个 ByteBuf 聚合成一个逻辑视图，**不发生实际的数据搬移**，读取时像操作单个 Buffer 一样。

2）发送文件时，Netty 调用 JDK 的 `FileChannel.transferTo()` ，直接从文件系统缓存写到网卡，压根不经过用户态内存。这一步才是真正利用了 OS 的零拷贝能力，底层对应 `sendfile` 系统调用。

3）还有直接使用堆外内存（Direct Buffer），避免 JVM 堆内 GC 带来的复制开销。数据从网卡读取直接落进堆外内存，处理完也能直接发出去，中间不再拷贝到堆内 byte[]。

典型场景就是大文件传输或者协议拼包，像 Kafka、RocketMQ 都用 Netty 做传输层，靠的就是这种高效内存管理扛住高吞吐。



注意，很多面试者容易把“不用拷贝”理解成完全无复制，其实只是链路整体减少了冗余拷贝，关键路径上能省则省。

[简单说说 Netty 的线程模型，并且其是如何完成工作的？]

Netty 的线程模型基于 Reactor 模式，核心是 **EventLoop**。它用少量线程支撑高并发连接，每个 EventLoop 负责处理多个 Channel 的 I/O 事件和任务调度。

1）启动时，服务端创建两个 NioEventLoopGroup：boss 负责 accept 新连接，worker 负责 read/write 数据。
2）新连接接入后，boss 将 Channel 注册到某个 worker 上，这个 worker 会一直处理该连接后续的所有 I/O 操作，保证线程安全。
3）每个 EventLoop 内部是一个死循环，轮询就绪的 Channel 事件，处理 IO、执行任务队列中的 Runnable（比如 channel.write() 提交的任务）。

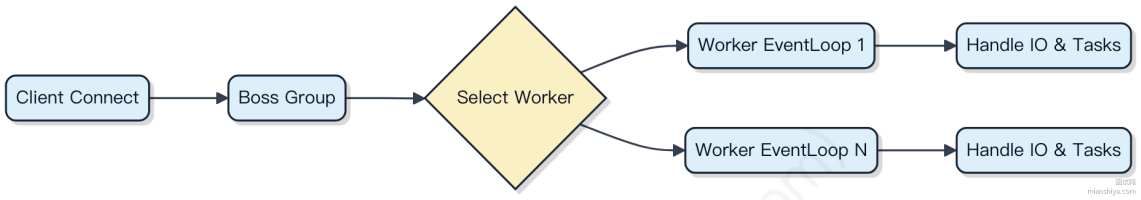
整个过程压根不经过额外线程切换，IO 和业务逻辑都在同一个 EventLoop 线程中串行执行，避免了锁竞争。但要注意，耗时操作不能直接在 handler 里做，否则会阻塞当前连接的事件处理。通常把耗时任务提交到自定义线程池。

举个例子，用 Netty 做网关时，decode 和路由转发可以放在 EventLoop，而调用下游服务这种网络请求就得扔进业务线程池，不然扛不住并发。

代码上体现为：

```
ServerBootstrap b = new ServerBootstrap();
b.group(bossGroup, workerGroup)
 .channel(NioServerSocketChannel.class)
 .childHandler(new ChannelInitializer<SocketChannel>() {
     protected void initChannel(SocketChannel ch) {
         ch.pipeline().addLast(new Decoder(), new BusinessHandler());
     }
 });
```

BusinessHandler 中若涉及慢操作，就得自己 manage 线程。



为什么不选择使用原生的 NIO 而选择使用 Netty 呢？

直接上 NIO 其实不是搞不定，但开发效率和维护成本会高到离谱。NIO 的 API 本身太底层了，像 ByteBuffer 的手动管理、粘包拆包处理、各种 Channel 的状态维护，全是脏活累活。

Netty 的核心优势是把网络编程里的通用问题都封装好了。比如它用 **ByteBuf** 重新设计了缓冲区，支持池化、自动扩容，还带引用计数，内存管理比原生的 Buffer 好用太多。

粘包问题 Netty 提供了现成的解码器，像 `LengthFieldBasedFrameDecoder` ，几行配置就能搞定，而原生 NIO 得自己写一大堆逻辑判断边界。

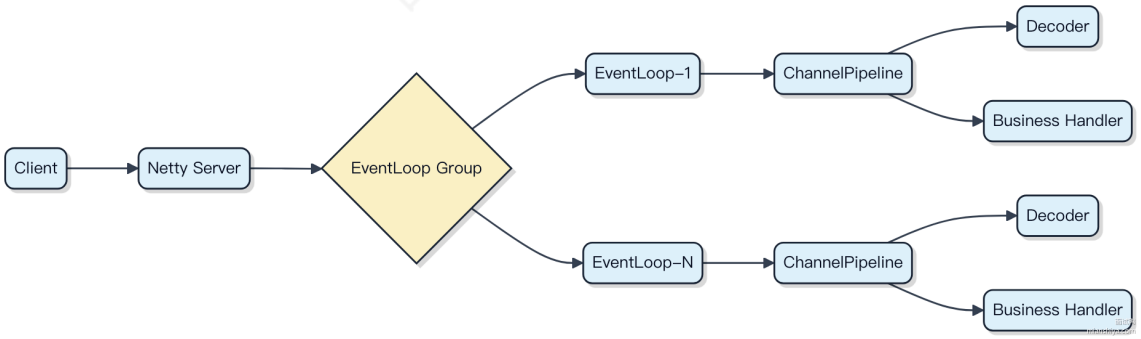
线程模型上，Netty 封装了 EventLoop，把 Reactor 模型玩得很透。一个 Channel 绑定一个 EventLoop，线程安全直接保障，不用操心并发冲突。原生 NIO 虽然也能实现多路复用，但 Reactor 的分发逻辑全得自己写，稍微复杂点就容易出错。

代码示例简单看个服务端启动：

```
new ServerBootstrap()
    .group(bossGroup, workerGroup)
    .channel(NioServerSocketChannel.class)
    .childHandler(new ChannelInitializer<SocketChannel>() {
        protected void initChannel(SocketChannel ch) {
            ch.pipeline().addLast(new MyBusinessHandler());
        }
    })
    .bind(8080);
```

这几十行代码背后，Netty 已经帮你处理了连接建立、事件轮询、异常传播、资源释放等全流程。

1）NIO 适合学习原理，但生产开发效率太低
2）Netty 提供了开箱即用的编解码、心跳、流量整形等能力
3）社区生态强，主流中间件如 Dubbo、RocketMQ 都基于它做通信层



Netty 性能为什么这么高？

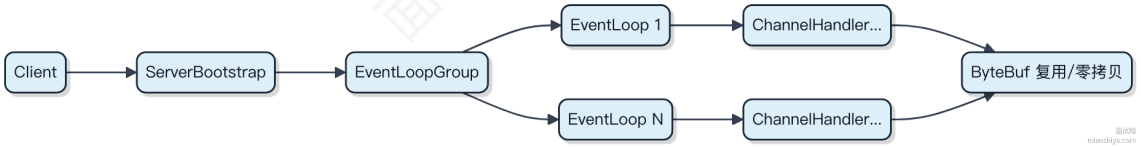
Netty 的高性能不是靠单一技术堆出来的，而是从线程模型、内存管理到 API 设计一整套协同优化的结果。

Reactor 模式是它底层的主心骨。服务端用少量线程就能同时处理成千上万的连接，不像传统 BIO 每个连接起一个线程，根本扛不住高并发。Netty 的 EventLoopGroup 把线程和事件循环包得干净利落，连接的读写事件全由固定的线程负责，避免上下文切换开销。

内存层面，对象池化玩得很深。ByteBuf 可以复用，减少 GC 压力。一次请求的数据缓冲如果每次都 new，GC 频繁时系统直接卡住。Netty 提供 PooledByteBufAllocator，默认情况下能复用内存块，吞吐量提升明显。

零拷贝不只是“不拷贝”，而是尽量让数据传输路径最短。比如 CompositeByteBuf 可以把多个 buffer 聚合成一个逻辑整体，发送时不用合并复制。文件传输用 FileRegion，直接走 JVM 之外的通道，压根不经过用户内存。

代码写起来也省心，责任链式的 ChannelPipeline 让业务逻辑拆解清晰，每个 Handler 各司其职。但别小看这设计，它保证了扩展性的同时，性能损耗几乎可以忽略。



对比原生 NIO，Netty 封装了繁琐的 register、select、dispatch 流程，API 简洁不说，还避开了很多生产环境的坑，比如 epoll 空轮询导致 CPU 100%。这种细节处理，才是它能在 Dubbo、RocketMQ、gRPC 里站稳脚跟的原因。

说一下 TCP 半包和粘包问题？

TCP 半包和粘包问题，本质是 TCP 协议设计本身决定的。TCP 是面向字节流的协议，它不关心应用层数据的边界，只保证字节流按序到达。所以发送方发了两条消息，接收方可能一次读到两条（粘包），也可能分两次读到一条消息的两半（半包）。

解决这个问题，关键在**应用层自己定义消息边界**。常见的做法有几种：

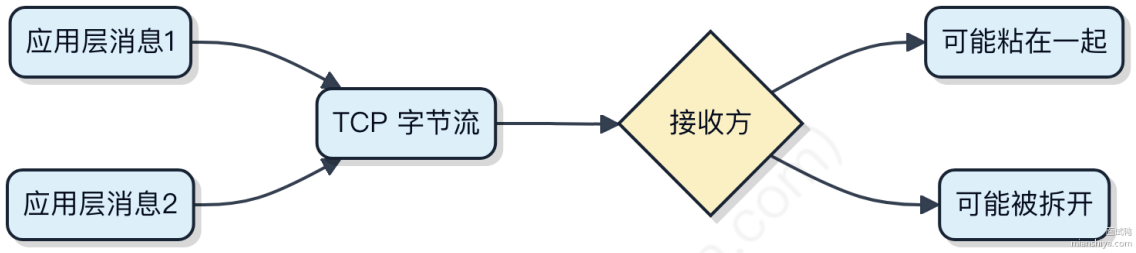
1）固定长度：每个消息都是 100 字节，不够补零。简单粗暴，但浪费带宽。

2）特殊分隔符：比如用 `\n` 或 `$$$` 结尾，类似 HTTP 头。Redis 的 RESP 协议就这么干的。

3）带长度前缀：最常用。先写 4 字节 int 表示 body 长度，再写实际数据。Netty 里直接用 `LengthFieldBasedFrameDecoder` 就能搞定。

```
// 伪代码示意：解码时先读 length，再读对应字节数
int length = in.readInt();
byte[] data = new byte[length];
in.readBytes(data);
```

像 Netty、Dubbo 这些框架，底层都内置了这类编解码器。你要是自己写 NIO 程序，没处理这个，基本搞不定可靠通信。



压根不经过应用层干预的话，数据就乱套了。所以别指望 TCP 自己会分包，脏活累活得自己接。

Netty 是如何解决粘包和拆包问题的？

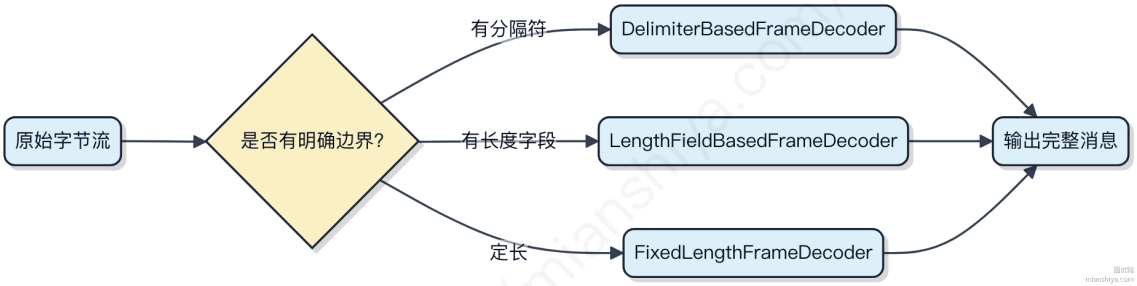
TCP 本身是面向流的协议，不关心应用层消息边界，这就导致接收方可能读到半条消息（拆包）或者多条消息粘在一起（粘包）。Netty 不是“解决”这个问题，而是提供了一组**解码器工具类**来帮助我们定义消息边界。

1）最常见的办法是加长度字段。比如用 `LengthFieldBasedFrameDecoder` ，它会先读取一个表示 body 长度的字段，再按这个长度截取完整报文。像 Dubbo、RocketMQ 内部都这么干，头部有个固定的 length 字段。

2）如果协议简单，可以用分隔符。比如每条消息结尾加 `\n` 或自定义分隔符，对应 Netty 的 `DelimiterBasedFrameDecoder` 。适合文本协议，比如 Redis 的 RESP 协议早期版本就靠回车换行切分。

3）定长消息直接用 `FixedLengthFrameDecoder` ，每个包固定大小。适用于心跳包这种场景，比如每 30 秒发一个 20 字节的心跳。

其实关键就是：**你得自己定义协议格式，让接收方能识别一条完整消息从哪开始、到哪结束**。Netty 提供这些开箱即用的工具，避免我们手动去缓冲、拼接字节流。



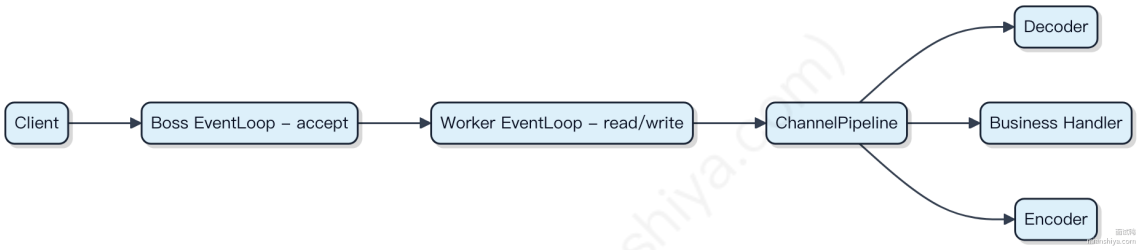
Netty 采用了哪些设计模式？

Netty 的设计里，Reactor 模式是主心骨。它用事件循环（EventLoop）来统一调度 I/O 读写和任务执行，把网络通信的复杂性屏蔽掉，让业务逻辑可以专注处理数据。服务端启动时绑定一个或多个 EventLoopGroup，每个连接由固定的 EventLoop 处理，避免锁竞争。

1）单线程 Reactor：所有操作都在一个线程完成，适合低并发测试。 2）多线程 Reactor：I/O 线程只负责读写，Handler 逻辑扔给业务线程池，提升吞吐。 3）主从 Reactor：Boss 线程监听 accept，Worker 线程池处理 read/write，生产环境常用。

责任链模式体现在 ChannelPipeline 上。每个 ChannelHandler 是一环，入站出站事件沿链传递。你可以动态添加日志、编解码、限流等处理器，彼此解耦。比如加个 `StringDecoder` 和 `LoggingHandler` 完全不用改原有逻辑。

工厂模式藏在 ByteBuf 分配里。通过 `ByteBufAllocator` 统一创建堆内/堆外内存缓冲区，上层代码不关心具体实现。PooledUnsafeDirectByteBuf 和 UnpooledHeapByteBuf 各司其职，复用内存池减少 GC 压力。



别小看这些模式的组合，它们让 Netty 既灵活又扛住高并发。像 RocketMQ、Dubbo 底层都靠这套机制撑着。

Netty 如何解决 JDK NIO 中的空轮询 Bug?

Netty 并没有从底层修复 JDK NIO 的 epoll 空轮询 Bug，而是通过**上层机制规避**了它带来的影响。这个 Bug 表现在某些 Linux 版本的 JDK 中，Selector 会在没有实际 I/O 事件的情况下返回，导致 CPU 占用飙到 100%。

Netty 的解决方案其实很务实：它给 Selector 设置了一个“**最大连续空轮询次数**”阈值，默认是 512 次。每发生一次空轮询，计数器加 1。一旦达到这个阈值，Netty 就会重建 Selector。

重建 Selector 是关键操作。它把所有 Channel 重新注册到新的 Selector 上，绕过旧 Selector 可能存在的内核 bug 状态。这个过程虽然有点重，但远比持续空跑耗 CPU 强。

代码层面，这个逻辑在 `NioEventLoop` 的 `select` 循环里：

```
if (selectedKeys != 0 || oldWakenUp || wakenUp.get() || hasTasks() ||
hasScheduledTasks()) {
    // 有事件或任务，正常处理
} else {
    // 空轮询，计数++
    if (selectedCnt >= SELECTOR_AUTO_REBUILD_THRESHOLD) { // 默认512
        rebuildSelector(); // 重建
        return;
    }
}
```

这种做法不依赖 JDK 修复，兼容性强，像 Dubbo、RocketMQ 这些用 Netty 做通信层的框架，都间接受益于这个防护机制。

要不要触发重建，还得看实际负载。如果真没请求，有任务调度也不会进空轮询。只有死循环无意义唤醒才算。

在 Netty 中，什么是 Channel？ 什么是 ChannelHandlerContext?

Channel 是 Netty 中网络通信的入口，代表了一个连接，比如 TCP 连接。它负责读写数据、绑定端口、关闭连接这些操作。每个客户端连接到服务器时，Netty 都会创建一个 Channel 实例，你可以把它看作是对 Socket 的增强封装。

ChannelHandlerContext 则是 ChannelHandler 在 pipeline 中的位置上下文。它持有 handler 和当前节点的引用，用来实现事件在 pipeline 中的流动。你通过 ctx 读写数据，其实走的是 pipeline 的拦截器链，而不是直接操作 Channel。

两者关键区别在于职责分离。Channel 是面向连接的统一抽象，而 ChannelHandlerContext 是 handler 在处理链中的“坐标”，控制事件传播方向。比如你调用 `ctx.write(data)` ，数据会从当前 handler 往前或往后传，但直接调用 `channel.write(data)` 就是从 tail 开始逆向传播。

常见误区是混用 ctx 和 channel 发送消息。一般建议用 ctx，粒度更细，性能更好，避免绕路。

1）Channel 负责底层 I/O 操作
2）ChannelHandlerContext 管控事件在 pipeline 中的流向
3）同一个 handler 里拿到的 ctx 是唯一的，生命周期与 Channel 一致

用 Netty 写服务端时，几乎每天都在和这两个东西打交道，搞清它们的关系，才能把编解码、心跳、超时这些脏活累活安排明白。

> *本资源来自面试鸭：https://www.mianshiya.com*
>
> *推荐更多免费学编程资源：*
>
> 1. *编程导航学习网站：学编程、做项目、拿 Offer！*
> 2. *企业高频面试题库：开始刷题，面试遇原题！*
> 3. *精选简历模板大全：1 分钟搞定简历！*
> 4. *AI 资源导航网站：获取最新 AI 黑科技！*
> 5. *1 对 1 模拟面试：随时随地提升面试能力*